

claude-windows / 42211f80-2169-4025-ad9d-99c1aa29ff70

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\42211f80-2169-4025-ad9d-99c1aa29ff70\subagents\workflows\wf_d4359b2a-456\agent-aeefaaec6a2be5ee.jsonl`  
- SHA-256: `107fae53f4f1fe02da2dc8fd737d0f86da5095ff36842df858e7f96a0101c1d1`  
- Source modified: `2026-06-06T15:22:27+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_d4359b2a-456`  
- session_id: `42211f80-2169-4025-ad9d-99c1aa29ff70`
```

Transcript

1. user (2026-06-06T15:20:37.519Z)

You are mapping ONE subsystem of the "badminton-highlight-indexer" Python repo (cwd is the repo root) for a dense CODE MAP that a future AI engineer will read to ramp up FAST on the codebase without crawling files.

Subsystem: Detectors & windowing

Files: backend/detectors/wasb_infer.py, backend/detectors/wasb_runner.py, backend/detectors/tracknet_runner.py, backend/detectors/__init__.py. PAY SPECIAL ATTENTION to parse_trajectory_csv + trajectory_to_action_windows (the model-agnostic windowing brain, defined on TrackNetRunner and reused by wasb) and to the resumable detector cache in wasb_infer.py (det_raw.jsonl, manifest, load_and_clean_cache, default_cache_dir, atomic writes).

READ each listed file (use Read; use Grep/Glob to resolve any directory globs to actual files). Then return, per file:

- path (exact, repo-relative, forward slashes)
- role: one terse line ? what the file is for
- symbols: the IMPORTANT classes / functions / dataclasses / module-level constants & registries. For each: name, kind (class|func|dataclass|const|registry), a terse signature (params + return where useful), and a one-line role. SKIP trivial private helpers and boilerplate.
- contracts: exact data shapes this file defines or consumes (dict keys, CSV column orders, tuple layouts, dataclass fields, return types). Be precise ? these are the cross-module interfaces.
- edges: calls/imports to OTHER files that matter for dataflow, each as "thisSymbol -> otherFile.thing (why)".
- gotchas: non-obvious constraints, footguns, platform/WSL/GPU/threadpool specifics, dead paths, defaults that bite.

Rules: be terse and ACCURATE. Every symbol and path MUST actually exist in the file ? do NOT invent or guess. If something is notable but you're unsure, omit it. Return via the structured schema.

2. assistant (2026-06-06T15:20:40.386Z)

I'll map the "Detectors & windowing" subsystem. Let me start by reading all the listed files.

3. user (2026-06-06T15:20:41.448Z)

```
1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).  
2  
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its  
4      own building blocks (ImageDataset transform + detector + online tracker) to run on  
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the  
6      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:  
7
```

```

8         Frame, Visibility, X, Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36     python wasb_infer.py --frames_dir /path/frames --weights
37     ../pretrained_weights/wasb_badminton_best.pth.tar \
38     --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39     every 1000] [--fresh]
40
41     """
42     import os
43     import os.path as osp
44     import csv
45     import glob
46     import json
47     import argparse
48     from collections import defaultdict
49
50     CACHE_VERSION = 1
51     _DET_FILE = "det_raw.jsonl"
52     _MANIFEST_FILE = "manifest.json"
53
54     def _build_cfg(weights: str, sport: str):
55         """Compose the WASB eval config via Hydra, overriding model/weights/device."""
56         from hydra import compose, initialize
57         with initialize(config_path="configs", version_base=None):
58             cfg = compose(config_name="eval", overrides=[
59                 f"dataset={sport}",
60                 "model=wasb",
61                 f"detector.model_path={weights}",
62                 "runner.device=cuda",
63                 "runner.gpus=[0]", # this box has a single GPU; default config assumes
64                 ])
65         return cfg
66
67     def _frame_id(path: str) -> int:
68         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
69         180)."""
70         stem = osp.splitext(osp.basename(path))[0]

```

```

69         digits = "".join(ch for ch in stem if ch.isdigit())
70         return int(digits) if digits else -1
71
72
73     # ----- #
74     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
75     # ----- #
76     def _cache_paths(cache_dir: str):
77         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
78
79
80     def default_cache_dir(out_csv: str) -> str:
81         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
82         d = osp.dirname(osp.abspath(out_csv))
83         stem = osp.splitext(osp.basename(out_csv))[0]
84         return osp.join(d, stem + "_wasbcache")
85
86
87     def _atomic_write_text(path: str, text: str) -> None:
88         """Write then rename so a crash mid-write can't leave a half-written file."""
89         tmp = path + ".tmp"
90         with open(tmp, "w") as f:
91             f.write(text)
92             f.flush()
93             os.fsync(f.fileno())
94         os.replace(tmp, path)
95
96
97     def _det_plain(det) -> list:
98         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
99         list."""
100         xy = det["xy"]
101         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
102
103     def _dets_to_tracker(plain_list):
104         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
105         array,
106         the tracker does vector math / np.linalg.norm on it)."""
107         import numpy as np
108         return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
109                 for p in plain_list]
110
111     def write_manifest(cache_dir: str, manifest: dict) -> None:
112         _, mpath = _cache_paths(cache_dir)
113         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
114
115
116     def read_manifest(cache_dir: str):
117         _, mpath = _cache_paths(cache_dir)
118         if not osp.exists(mpath):
119             return None
120         try:
121             with open(mpath) as f:
122                 return json.load(f)
123         except (json.JSONDecodeError, OSError):
124             return None
125
126
127     def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
128                             frames_in: int, frames_total: int) -> bool:
129         """A cache is reusable only if the run that produced it matches this run's
130         inputs."""
131         if not manifest or manifest.get("version") != CACHE_VERSION:

```

```

131         return False
132     return (
133         manifest.get("frames_dir") == osp.abspath(frames_dir)
134         and manifest.get("weights") == weights
135         and manifest.get("sport") == sport
136         and manifest.get("frames_in") == frames_in
137         and manifest.get("frames_total") == frames_total
138     )
139
140
141 def reset_cache(cache_dir: str) -> None:
142     """Drop any existing cache so the next run starts clean."""
143     det_jsonl, mpath = _cache_paths(cache_dir)
144     for p in (det_jsonl, mpath):
145         if osp.exists(p):
146             os.remove(p)
147
148
149 def load_and_clean_cache(cache_dir: str):
150     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
151     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
152     prefix so a trailing half-written line from a crash can't corrupt future appends.
153
154     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
155     [x, y, score, scale] candidates (accumulated across every window the frame is in,
156     in window order ? identical to a non-resumed run).
157     """
158     det_jsonl, _ = _cache_paths(cache_dir)
159     det_by_fid = defaultdict(list)
160     valid = []
161     if osp.exists(det_jsonl):
162         with open(det_jsonl, "r") as f:
163             for line in f:
164                 s = line.strip()
165                 if not s:
166                     continue
167                 try:
168                     obj = json.loads(s)
169                 except json.JSONDecodeError:
170                     break # truncated trailing line (crash mid-
flush)
171                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
172                     break
173                 valid.append(s)
174                 for fid, plain in obj["f"]:
175                     det_by_fid[int(fid)].extend([list(p) for p in plain])
176                 # Guarantee a clean append boundary by rewriting only the good prefix.
177                 _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
178     return len(valid), det_by_fid
179
180
181 def _append_windows(fh, objs) -> None:
182     """Append window records as JSONL and durably flush (bounded loss on crash)."""
183     for o in objs:
184         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
185     fh.flush()
186     os.fsync(fh.fileno())
187
188
189 def _atomic_write_trajectory(out_csv: str, rows) -> None:
190     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
191     tmp = out_csv + ".tmp"
192     with open(tmp, "w", newline="") as f:
193         w = csv.writer(f)
194         w.writerow(["Frame", "Visibility", "X", "Y"])

```

```

195         w.writerows(rows)
196         f.flush()
197         os.fsync(f.fileno())
198         os.replace(tmp, out_csv)
199
200
201     # ----- #
202     # Inference
203     # ----- #
204     def run(frames_dir: str, weights: str, out_csv: str, sport: str = "badminton",
205            limit: int = 0, batch_size: int = 8, num_workers: int = 4,
206            log_every_batches: int = 50, cache_dir: str = None,
207            flush_every: int = 1000, fresh: bool = False) -> str:
208         import time
209         import torch
210         from torch.utils.data import DataLoader
211         from detectors import build_detector
212         from trackers import build_tracker
213         from dataloaders import build_img_transforms, build_seq_transforms
214         from dataloaders.dataset_loader import ImageDataset
215         from utils import Center
216
217         cfg = _build_cfg(weights, sport)
218         frames_in = int(cfg["model"]["frames_in"])
219         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
220         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
221
222         frames = sorted(glob.glob(osp.join(frames_dir, "*.png")) +
223                        glob.glob(osp.join(frames_dir, "*.jpg")))
224         if limit:
225             frames = frames[:limit]
226         if len(frames) < frames_in:
227             raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in
228 {frames_dir}")
229
230         # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
231 checkpoint).
232         samples = []
233         for i in range(len(frames) - frames_in + 1):
234             window = frames[i:i + frames_in]
235             annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
236                     for p in window]
237             samples.append({"frames": window, "annos": annos})
238         windows_total = len(samples)
239
240         # ---- cache setup / resume decision ----- #
241         if cache_dir is None:
242             cache_dir = default_cache_dir(out_csv)
243             os.makedirs(cache_dir, exist_ok=True)
244             det_jsonl, _ = _cache_paths(cache_dir)
245             manifest = None if fresh else read_manifest(cache_dir)
246             resume = manifest_compatible(
247                 manifest or {}, frames_dir=frames_dir, weights=weights, sport=sport,
248                 frames_in=frames_in, frames_total=len(frames),
249             )
250         if resume:
251             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
252             print(f"[wasb_infer] resuming from cache: {windows_done}/{windows_total} windows
253 already detected",
254                 flush=True)
255         else:
256             if manifest is not None:
257                 print("[wasb_infer] cache incompatible with this run -> starting fresh",
258                     flush=True)
259             reset_cache(cache_dir)

```

```

254         windows_done, det_by_fid = 0, defaultdict(list)
255
256     base_manifest = {
257         "version": CACHE_VERSION,
258         "frames_dir": osp.abspath(frames_dir),
259         "weights": weights,
260         "sport": sport,
261         "frames_in": frames_in,
262         "frames_total": len(frames),
263         "windows_total": windows_total,
264         "windows_done": windows_done,
265     }
266     write_manifest(cache_dir, base_manifest)
267
268     # ---- detector pass over the un-cached windows ----- #
269     remaining = samples[windows_done:]
270     if remaining:
271         _, transform_test = build_img_transforms(cfg)
272         try:
273             _, seq_transform_test = build_seq_transforms(cfg)
274         except Exception:
275             seq_transform_test = None
276         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
277                          transform=transform_test, seq_transform=seq_transform_test,
278                          is_train=False)
279         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
280                             num_workers=num_workers)
281         detector = build_detector(cfg)
282
283         t0 = time.time()
284         done = windows_done
285         win_idx = windows_done
286         log_every = max(1, log_every_batches)
287         flush_n = max(1, flush_every)
288         pending = []
289         print(f"[wasb_infer] detecting on {len(remaining)} remaining windows "
290               f"(total {windows_total}, batch={batch_size}, workers={num_workers})...",
291               flush=True)
292         det_f = open(det_jsonl, "a")
293         try:
294             with torch.no_grad():
295                 for bi, batch in enumerate(loader):
296                     imgs, _hms, trans = batch[0], batch[1], batch[2]
297                     img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
298                     path
299
300                     batch_results, _ = detector.run_tensor(imgs, trans)
301                     nb = imgs.shape[0]
302                     for ib in range(nb):
303                         frames_payload = []
304                         for ie in sorted(batch_results[ib].keys()):
305                             p = img_paths[ie][ib]
306                             fid = _frame_id(p)
307                             plain = [_det_plain(d) for d in batch_results[ib][ie]]
308                             frames_payload.append([fid, plain])
309                             det_by_fid[fid].extend(plain)
310                         pending.append({"w": win_idx, "f": frames_payload})
311                         win_idx += 1
312                     done += nb
313                     if len(pending) >= flush_n or done >= windows_total:
314                         _append_windows(det_f, pending)
315                         pending = []
316                         base_manifest["windows_done"] = done
317                         write_manifest(cache_dir, base_manifest)
318                     if (bi + 1) % log_every == 0 or done >= windows_total:
319                         el = time.time() - t0

```

```

315         new_done = done - windows_done
316         rate = new_done / el if el > 0 else 0.0
317         eta = (windows_total - done) / rate if rate > 0 else 0.0
318         print(f"[wasb_infer] {done}/{windows_total} "
319               f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
320               f"elapsed {el:.0f}s | ETA {eta:.0f}s", flush=True)
321         if pending:
322             _append_windows(det_f, pending)
323             base_manifest["windows_done"] = done
324             write_manifest(cache_dir, base_manifest)
325         finally:
326             det_f.close()
327             print(f"[wasb_infer] detection done in {time.time() - t0:.0f}s; running
tracker...", flush=True)
328         else:
329             print("[wasb_infer] detector cache already complete -> tracker-only re-run",
flush=True)
330
331         # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
332         tracker = build_tracker(cfg)
333         tracker.refresh()
334         rows = []
335         for fid in sorted(det_by_fid.keys()):
336             r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
337             rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
338
339         _atomic_write_trajectory(out_csv, rows)
340         visible = sum(1 for _, v, *_ in rows if v)
341         print(f"[wasb_infer] wrote {len(rows)} rows -> {out_csv}")
342         print(f"[wasb_infer] visible-shuttle frames: {visible}/{len(rows)}")
343         return out_csv
344
345
346     def extract_frames(video_path: str, frames_dir: str) -> str:
347         """Decode a video to PNG frames named by source frame index (00000000.png ...).
348
349         Idempotent: if the dir already holds the expected number of frames (matching the
350         container's reported frame count) we skip re-decoding ? re-extraction after a
351         reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
352         extracting with ffmpeg on the host and passing --frames_dir.
353         """
354         import cv2
355         os.makedirs(frames_dir, exist_ok=True)
356         cap = cv2.VideoCapture(video_path)
357         if not cap.isOpened():
358             raise SystemExit(f"cannot open video: {video_path}")
359         expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
360         existing = len(glob.glob(osp.join(frames_dir, "*.png")))
361         if expected > 0 and existing >= expected:
362             cap.release()
363             print(f"[wasb_infer] reusing {existing} already-extracted frames ->
{frames_dir}")
364             return frames_dir
365         i = 0
366         while True:
367             ok, frame = cap.read()
368             if not ok:
369                 break
370             cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
371             i += 1
372         cap.release()
373         print(f"[wasb_infer] extracted {i} frames -> {frames_dir}")
374         return frames_dir
375

```

```

376
377 def main():
378     ap = argparse.ArgumentParser()
379     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
380     ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
381     ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
382     ap.add_argument("--weights", required=True)
383     ap.add_argument("--out", required=True)
384     ap.add_argument("--sport", default="badminton")
385     ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
386     ap.add_argument("--batch-size", type=int, default=8)
387     ap.add_argument("--num-workers", type=int, default=4)
388     ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
389     ap.add_argument("--cache-dir", default=None,
390                     help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
391     ap.add_argument("--flush-every", type=int, default=1000,
392                     help="fsync the detector cache every N windows (bounds reboot
loss)")
393     ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
394     args = ap.parse_args()
395
396     frames_dir = args.frames_dir
397     if args.video:
398         frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
399     if not frames_dir:
400         raise SystemExit("provide --frames_dir or --video")
401     run(frames_dir, args.weights, args.out, args.sport, args.limit,
402         batch_size=args.batch_size, num_workers=args.num_workers,
403         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
404         flush_every=args.flush_every, fresh=args.fresh)
405
406
407 if __name__ == "__main__":
408     main()
409

```

4. user (2026-06-06T15:20:41.850Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     windows-side counterpart to backend/detectors/wasb_infer.py. It:
4     1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5     2. copies wasb_infer.py into the WASB repo's src/ dir,
6     3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7     4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on

```



```

TrackNetRunner.
21     from backend.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22
23     logger = logging.getLogger(__name__)
24
25     # Windows path to the inference wrapper we stage into WSL.
26     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
27
28
29     @dataclass
30     class WasbConfig:
31         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
32         distro: str = "Ubuntu"
33         repo_dir: str = "~/models/WASB-SBDT"
34         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
35         conda_env: str = "wasb"
36         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
37         sport: str = "badminton"
38         wsl_stage_dir: str = "~/clips"
39         timeout_sec: int = 0 # 0 = no timeout
40
41         @classmethod
42         def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
43             w = (idx_cfg or {}).get("wasb", {}) or {}
44             return cls(
45                 distro=w.get("wsl_distro", "Ubuntu"),
46                 repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
47                 conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
48                 conda_env=w.get("conda_env", "wasb"),
49                 weights_path=w.get("weights_path",
~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
51                 sport=w.get("sport", "badminton"),
52                 wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
53                 timeout_sec=int(w.get("timeout_sec", 0)),
54             )
55
56
57     class WasbRunner:
58         def __init__(self, cfg: WasbConfig):
59             self.cfg = cfg
60
61         def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
62             full = f"source {self.cfg.conda_sh} && {inner_cmd}"
63             argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
64             logger.debug("WSL exec: %s", full)
65             return subprocess.run(argv, capture_output=True, text=True,
66                                 timeout=(self.cfg.timeout_sec or None))
67
68         def healthcheck(self) -> tuple:
69             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
70             cmd = (f"conda activate {self.cfg.conda_env} && "
71                  f"python -c \"import torch; print('torch', torch.__version__, "
72                  f"'cuda', torch.cuda.is_available())\"")
73             try:
74                 res = self._wsl_bash(cmd)
75             except FileNotFoundError:
76                 return False, "`wsl` not found ? Windows + WSL2 required."
77             except subprocess.TimeoutExpired:
78                 return False, "WSL healthcheck timed out."
79             out = (res.stdout or "").strip()
80             if res.returncode != 0:
81                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"

```

```

82         return True, out
83
84     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
85         src_mnt = to_wsl_mnt_path(video_win_path)
86         dest = f"{self.cfg.wsl_stage_dir}/{base}"
87         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)} {shlex.quote(dest)} && echo OK"
88         try:
89             res = self._wsl_bash(cmd)
90         except subprocess.TimeoutExpired:
91             logger.error("Staging video into WSL timed out.")
92             return None
93         if res.returncode != 0 or "OK" not in (res.stdout or ""):
94             logger.error("Failed to stage video into WSL: %s", (res.stderr or res.stdout).strip())
95             return None
96         return dest
97
98     def _stage_infer_script(self) -> bool:
99         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
100        src_mnt = to_wsl_mnt_path(_INFER_WIN)
101        cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo OK"
102        res = self._wsl_bash(cmd)
103        return res.returncode == 0 and "OK" in (res.stdout or "")
104
105    def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
106        """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None."""
107        os.makedirs(output_win_dir, exist_ok=True)
108        base = os.path.basename(video_win_path)
109        stem = os.path.splitext(base)[0]
110        wsl_out_dir = to_wsl_mnt_path(output_win_dir)
111        expected_csv_win = os.path.join(output_win_dir, f"{stem}_wasb.csv")
112
113        if not self._stage_infer_script():
114            logger.error("Could not stage wasb_infer.py into WSL.")
115            return None
116        staged_video = self._stage_video(video_win_path, base)
117        if staged_video is None:
118            return None
119        frames_dir = f"{self.cfg.wsl_stage_dir}/{stem}_frames"
120
121        cmd = (
122            f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
123            f"python wasb_infer.py "
124            f"--video {shlex.quote(staged_video)} "
125            f"--frames_out_dir {shlex.quote(frames_dir)} "
126            f"--weights {shlex.quote(self.cfg.weights_path)} "
127            f"--sport {shlex.quote(self.cfg.sport)} "
128            f"--out {shlex.quote(wsl_out_dir + '/' + stem + '_wasb.csv')}"
129        )
130        logger.info("Running WASB inference on %s ...", base)
131        try:
132            res = self._wsl_bash(cmd)
133        except subprocess.TimeoutExpired:
134            logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
135            return None
136        if res.returncode != 0:
137            logger.error("WASB inference failed:\n%s", (res.stderr or res.stdout)[-2000:])
138            return None
139        if not os.path.exists(expected_csv_win):
140            logger.error("WASB finished but expected CSV not found at %s", expected_csv_win)

```

```

141         return None
142         logger.info("WASB trajectory CSV: %s", expected_csv_win)
143         return expected_csv_win
144
145     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
146     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
147     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
148

```

5. user (2026-06-06T15:20:42.818Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6         1. Translate the Windows video path to its WSL-visible form (and optionally
7            stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     logger = logging.getLogger(__name__)
26
27
28     @dataclass
29     class TrackNetConfig:
30         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
31         distro: str = "Ubuntu"
32         repo_dir: str = "~/models/TrackNetV4" # WSL path to the cloned repo
33         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
34         conda_env: str = "TrackNetV4"
35         weights_path: str = "" # WSL path to the .h5/.keras weights
36         (REQUIRED)
37         queue_length: int = 5
38         stage_in_wsl: bool = True # copy video into WSL fs first
39         (perf)
40         wsl_stage_dir: str = "~/clips" # where staged videos/outputs live
41         in WSL
42         timeout_sec: int = 0 # 0 = no timeout
43
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),

```

```

49         weights_path=tn.get("weights_path", ""),
50         queue_length=int(tn.get("queue_length", 5)),
51         stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
52         wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
53         timeout_sec=int(tn.get("timeout_sec", 0)),
54     )
55
56
57 def to_wsl_mnt_path(win_path: str) -> str:
58     """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
59
60     Already-POSIX paths (starting with / or ~) are returned unchanged so the
61     same helper is safe to call on either kind of path.
62     """
63     p = str(win_path)
64     if p.startswith("/") or p.startswith("~"):
65         return p
66     p = p.replace("\\", "/")
67     if len(p) >= 2 and p[1] == ":":
68         drive = p[0].lower()
69         return f"/mnt/{drive}{p[2:]}"
70     return p
71
72
73 @dataclass
74 class TrajectoryPoint:
75     frame: int
76     visible: bool
77     x: float
78     y: float
79
80
81 class TrackNetRunner:
82     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows."""
83
84     def __init__(self, cfg: TrackNetConfig):
85         self.cfg = cfg
86
87     # ----- WSL exec
88
89     def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
90         """Run a bash command string inside the configured WSL distro."""
91         full = f"source {self.cfg.conda_sh} && {inner_cmd}"
92         argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
93         logger.debug("WSL exec: %s", full)
94         return subprocess.run(
95             argv, capture_output=True, text=True,
96             timeout=(self.cfg.timeout_sec or None),
97         )
98
99     def healthcheck(self) -> Tuple[bool, str]:
100         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
101
102         Returns (ok, message). Cheap to call before a long run.
103         """
104         cmd = (
105             f"conda activate {self.cfg.conda_env} && "
106             f"python -c \"import tensorflow as tf; "
107             f"print('TF', tf.__version__, 'GPUs', "
108             f"len(tf.config.list_physical_devices('GPU')))\n"
109         )
110         try:
111             res = self._wsl_bash(cmd)
112         except FileNotFoundError:
113             return False, "`wsl` not found ? is this running on Windows with WSL2

```

```

installed?"
113         except subprocess.TimeoutExpired:
114             return False, "WSL healthcheck timed out."
115         out = (res.stdout or "").strip()
116         if res.returncode != 0:
117             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
118         return True, out
119
120     # ----- inference
121
122     def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
123         """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.
124
125         ``output_win_dir`` is a Windows directory (under the repo's output/) that
126         WSL writes into via its /mnt view, so the Windows process can read the CSV
127         back with no copy.
128         """
129         if not self.cfg.weights_path:
130             logger.error(
131                 "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
132                 "in config.json to the WSL path of your trained TrackNetV4 weights."
133             )
134             return None
135
136         os.makedirs(output_win_dir, exist_ok=True)
137         base = os.path.basename(video_win_path)
138         stem, ext = os.path.splitext(base)
139
140         # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
141         if ext.lower() not in (".mp4", ".avi"):
142             logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
143             return None
144
145         wsl_out = to_wsl_mnt_path(output_win_dir)
146         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
147
148         # Resolve the video path WSL will read.
149         if self.cfg.stage_in_wsl:
150             staged = self._stage_video(video_win_path, base)
151             if staged is None:
152                 return None
153             wsl_video = staged
154         else:
155             wsl_video = to_wsl_mnt_path(video_win_path)
156
157         cmd = (
158             f"conda activate {self.cfg.conda_env} && "
159             f"cd {self.cfg.repo_dir}/src && "
160             f"python predict.py "
161             f"--video_path {shlex.quote(wsl_video)} "
162             f"--model_weights {shlex.quote(self.cfg.weights_path)} "
163             f"--output_dir {shlex.quote(wsl_out)} "
164             f"--queue_length {self.cfg.queue_length}"
165         )
166         logger.info("Running TrackNet inference on %s ...", base)
167         try:
168             res = self._wsl_bash(cmd)
169         except subprocess.TimeoutExpired:
170             logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
171             return None
172
173         if res.returncode != 0:
174             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or

```

```

res.stdout)[-2000:])
175         return None
176
177         if not os.path.exists(expected_csv_win):
178             logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
179             return None
180             logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
181             return expected_csv_win
182
183     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
184         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
185
186         Returns the WSL path to the staged copy, or None on failure.
187         """
188         src_mnt = to_wsl_mnt_path(video_win_path)
189         dest = f"{self.cfg.wsl_stage_dir}/{base}"
190         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{shlex.quote(dest)} && echo OK"
191         try:
192             res = self._wsl_bash(cmd)
193         except subprocess.TimeoutExpired:
194             logger.error("Staging video into WSL timed out.")
195             return None
196         if res.returncode != 0 or "OK" not in (res.stdout or ""):
197             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
198             return None
199             return dest
200
201     # ----- CSV -> windows
202
203     @staticmethod
204     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
205         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
206         points: List[TrajectoryPoint] = []
207         with open(csv_win_path, newline="") as f:
208             reader = csv.DictReader(f)
209             for row in reader:
210                 try:
211                     points.append(TrajectoryPoint(
212                         frame=int(row["Frame"]),
213                         visible=int(row["Visibility"]) == 1,
214                         x=float(row["X"]),
215                         y=float(row["Y"]),
216                     ))
217                 except (KeyError, ValueError):
218                     continue
219             points.sort(key=lambda p: p.frame)
220             return points
221
222     @staticmethod
223     def trajectory_to_action_windows(
224         points: List[TrajectoryPoint],
225         fps: float,
226         frame_width: float,
227         chunk_start: float = 0.0,
228         velocity_thresh: float = 0.02,
229         merge_gap: float = 2.0,
230         min_window_duration: float = 2.0,
231         chunk_index: Optional[int] = None,
232     ) -> List[Dict[str, Any]]:
233         """Convert a shuttle trajectory into candidate rally windows.
234
235         A frame is "active" when the shuttle is visible AND its inter-frame

```

```

236 displacement (normalised by frame width, per second) exceeds
237 ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
238 ``merge_gap`` seconds of each other are grouped into one window; short
239 windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
240 shape feeding the AI-handoff phase is identical.
241 """
242 if fps <= 0:
243     fps = 30.0
244 if frame_width <= 0:
245     frame_width = 1280.0
246
247 # Compute per-frame normalised velocity from the last *visible* point.
248 active = [] # (frame_idx, velocity)
249 prev: Optional[TrajectoryPoint] = None
250 for p in points:
251     if not p.visible:
252         prev = None # break the trajectory; absent shuttle = not in flight
253         continue
254     if prev is not None:
255         dframes = p.frame - prev.frame
256         if dframes > 0:
257             dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
258             velocity = (dist / frame_width) * (fps / dframes)
259             if velocity > velocity_thresh:
260                 active.append((p.frame, velocity))
261     prev = p
262
263 if not active:
264     return []
265
266 max_gap_frames = int(merge_gap * fps)
267
268 def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
269     vels = [v for _, v in group]
270     return {
271         "start": group[0][0] / fps + chunk_start,
272         "end": group[-1][0] / fps + chunk_start,
273         "active_frame_count": len(group),
274         "peak_velocity": max(vels),
275         "mean_velocity": sum(vels) / len(vels),
276         "track_id_count": 1, # single shuttle; kept for window-shape parity
277         "chunk_index": chunk_index,
278     }
279
280 windows = []
281 group = [active[0]]
282 for af in active[1:]:
283     if af[0] - group[-1][0] <= max_gap_frames:
284         group.append(af)
285     else:
286         windows.append(_finalize(group))
287         group = [af]
288 windows.append(_finalize(group))
289
290 return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
291

```

6. user (2026-06-06T15:20:42.931Z)

```

1     """Low-level moving-object detectors that run outside the main Python process.
2
3     Currently houses the TrackNet WSL runner ? it shells out to a TensorFlow
4     TrackNetV4 environment living inside WSL2 (see docs/DECISIONS.md, ADR-001) and
5     returns the shuttle trajectory back to the Windows-side indexer. Keeping the
6     heavy TF dependency quarantined in WSL is deliberate: the main project env stays

```

```
7         free of TensorFlow/CUDA.
8         ""
9
```

7. assistant (2026-06-06T15:20:45.652Z)

All four files are read. I have what I need to produce the dense map. Let me return the structured output.

8. user (2026-06-06T15:22:14.234Z)

Structured output provided successfully

9. assistant (2026-06-06T15:22:27.662Z)

I've mapped the "Detectors & windowing" subsystem. The structured output covers all four files with precise contracts, edges, and gotchas. Key takeaways for a future engineer:

- **The model-agnostic windowing brain** (``parse_trajectory_csv` + `trajectory_to_action_windows``) lives as `@staticmethod`s` on ``TrackNetRunner`` in ``tracknet_runner.py`` and is re-exported verbatim by ``WasbRunner`` via ``staticmethod(TrackNetRunner.parse_trajectory_csv)``. Both detector backends therefore emit the identical action-window dict shape (``start, end, active_frame_count, peak_velocity, mean_velocity, track_id_count, chunk_index``) that downstream phases consume.
- **The resumable detector cache** in ``wasb_infer.py`` is the standout piece: it splits the expensive checkpointed GPU detector pass (``det_raw.jsonl`` JSONL, one window per line keyed ``w==line-index``, plus atomic ``manifest.json``) from the cheap, stateful, always-re-run CPU tracker pass. ``load_and_clean_cache`` heals crash-truncated tails by reading only the longest contiguous valid prefix and rewriting the file to it. Cache auto-invalidates on any input change via ``manifest_compatible``.
- **Two notable asymmetries worth flagging**: ``WasbRunner.run_predict`` does NOT forward the cache/batch flags to ``wasb_infer.py`` (relies on its CLI defaults), and ``_wsl_bash`` is duplicated across both runners rather than shared.

Relevant file paths (absolute):

- ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\detectors\tracknet_runner.py``
- ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\detectors\wasb_runner.py``
- ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\detectors\wasb_infer.py``
- ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\detectors\__init__.py``